Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

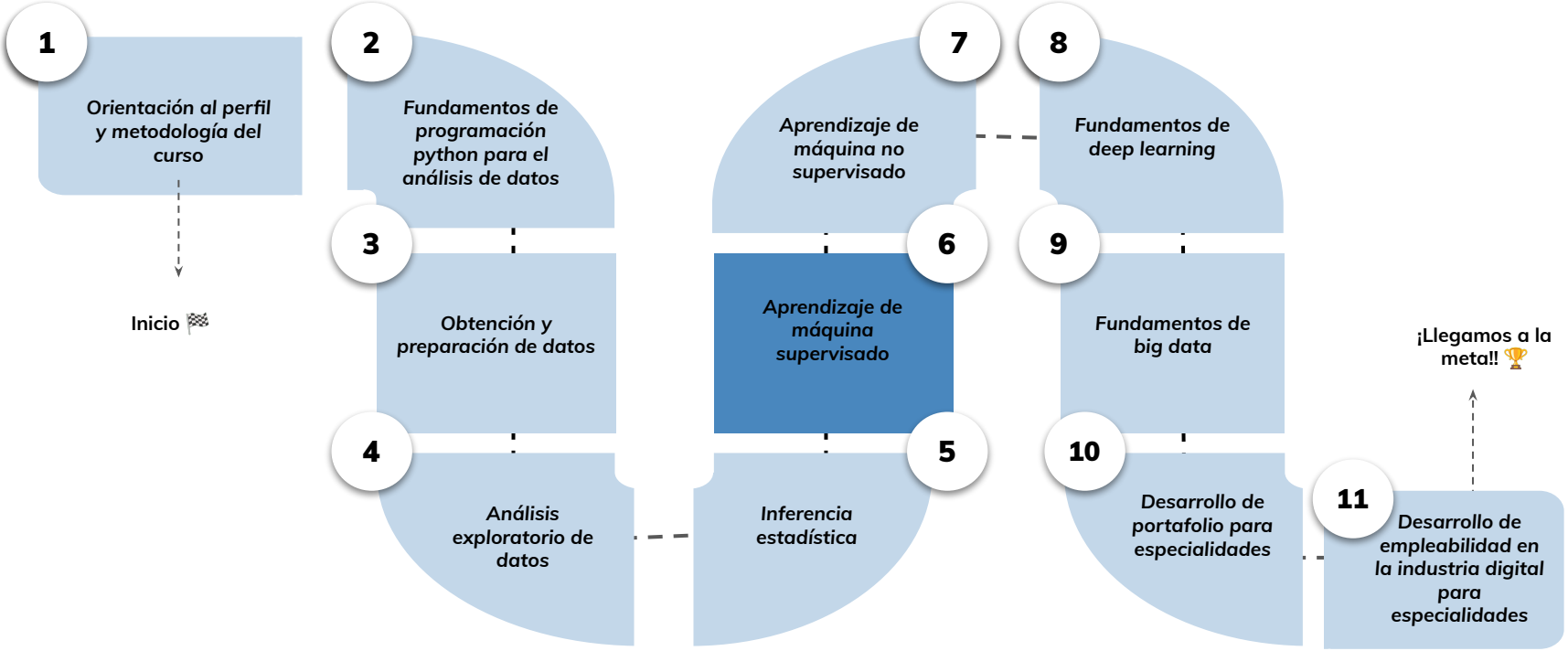
Nivel de ajuste de un modelo y validación cruzada- Parte I

Aprendizaje Esperado 2: Aplicar técnicas de validación cruzada para la prevención de sobreajuste en los modelos utilizando el concepto de nivel de ajuste óptimo.

alkemy

Hoja de ruta

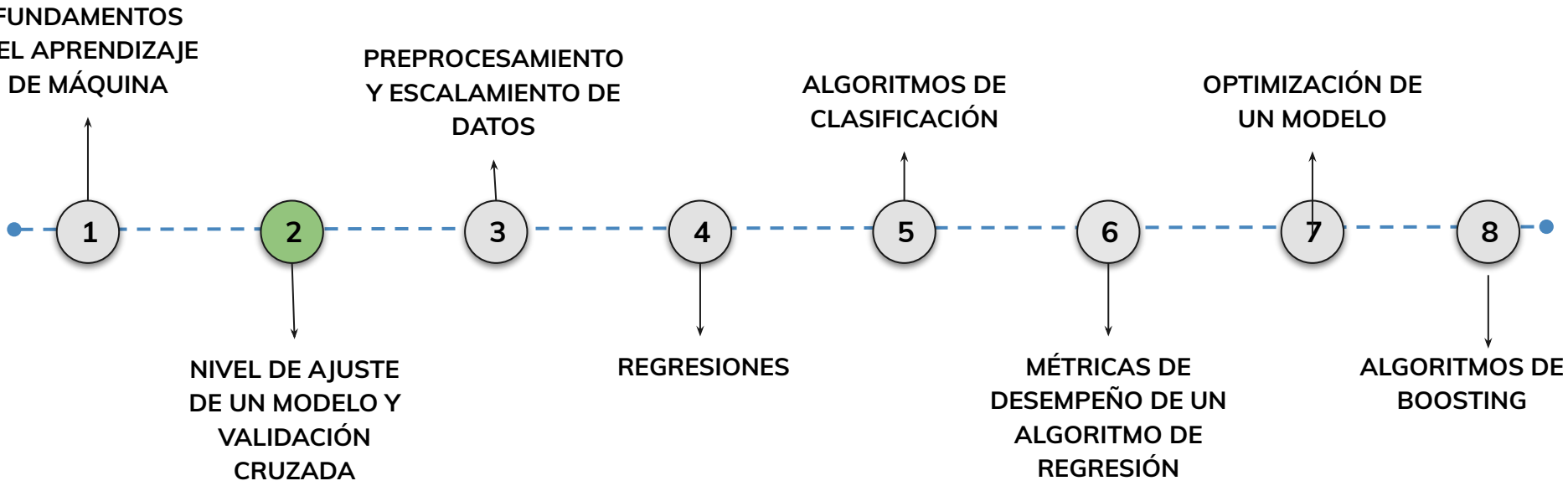
¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



alkemy

Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

2.

Nivel de ajuste de un modelo

hoy vamos a aprender qué significa que un modelo esté bien o mal ajustado, cómo evitar errores comunes y cómo evaluarlo correctamente.

- **Subajuste, sobreajuste y ajuste adecuado**
  - **Ejemplos visuales y diferencias clave**
  - **Impacto del sesgo y la varianza**
- **Validación cruzada**
  - **Qué es y para qué sirve**
  - **K-Fold y su implementación en Scikit-learn**

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- Identificamos cuándo un modelo está subajustado o sobreajustado
- Comprendimos el trade-off entre sesgo y varianza
- Reconocimos los distintos tipos de errores de un modelo
- Aplicamos validación cruzada para evaluar modelos
- Tomamos decisiones informadas sobre el nivel de ajuste óptimo

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

→ *Qué es un modelo de regresión*

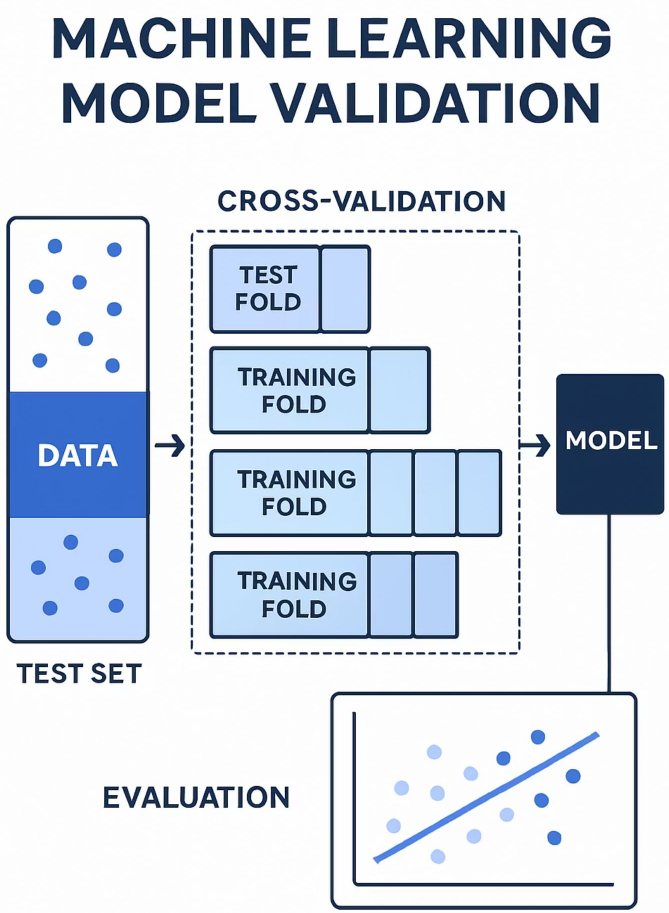
→ *Diferencias entre regresión y clasificación*

→ *Cómo entrenar un modelo de regresión lineal*

→ *Cómo evaluar el desempeño con métricas como R^2 y MAE*

alkemy

Nivel de ajuste de un modelo y validación cruzada



NIVEL DE AJUSTE DE UN MODELO Y VALIDACIÓN CRUZADA

alkemy

Introducción

En el mundo del aprendizaje de máquina, la construcción de un modelo predictivo no termina cuando el algoritmo ha sido entrenado. Una de las etapas más cruciales y, a menudo, subestimadas, es la evaluación del modelo y la comprensión de qué tan bien se ajusta a los datos. Los conceptos de ajuste, validación y evaluación son fundamentales para garantizar que un modelo no solo funcione bien en los datos conocidos, sino también en datos nuevos y no vistos.

El ajuste de un modelo hace referencia al grado en que la estructura y los parámetros del modelo representan la relación real entre las variables. Un ajuste inapropiado puede llevar a dos problemas comunes: el sobreajuste y el subajuste. Ambos fenómenos afectan directamente la capacidad del modelo para generalizar, es decir, para predecir correctamente sobre datos nuevos. Por esta razón, es necesario incorporar técnicas que permitan validar y medir el comportamiento del modelo bajo diferentes escenarios y conjuntos de datos.

¿Qué es el ajuste de un modelo?

El ajuste de un modelo es el proceso mediante el cual se entrena un algoritmo de aprendizaje de máquina para que pueda aprender patrones a partir de un conjunto de datos. Este ajuste implica encontrar un equilibrio entre la capacidad del modelo para explicar los datos de entrenamiento y su habilidad para generalizar a nuevos datos. Un modelo bien ajustado es aquel que aprende lo suficiente de los datos para hacer predicciones precisas, evitando tanto el sobreajuste como el subajuste.

Modelo demasiado complejo

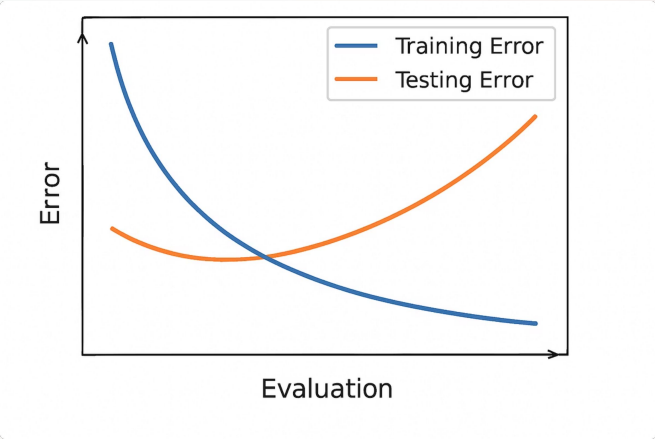
Si el modelo es demasiado complejo, corre el riesgo de aprender también el "ruido" o las particularidades del conjunto de entrenamiento, lo que lleva a un mal rendimiento con datos nuevos.

Modelo demasiado simple

Por otro lado, si el modelo es demasiado simple, es posible que no capture la estructura subyacente de los datos, lo que también afecta negativamente su desempeño.

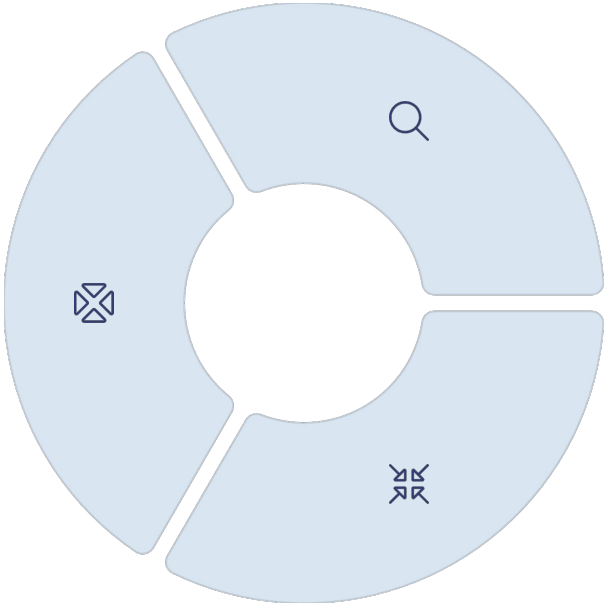
Medición del ajuste

Uno de los elementos esenciales para medir el ajuste es la diferencia entre el rendimiento en los datos de entrenamiento y en los datos de prueba. Cuando esta diferencia es pequeña y ambos resultados son buenos, se dice que el modelo tiene un ajuste apropiado. Sin embargo, si hay una gran diferencia, esto puede ser una señal de problemas.



alkemy

Posibles situaciones de ajuste



Ajuste adecuado

El modelo captura correctamente los patrones de los datos sin memorizar el ruido.

Sobreajuste

El modelo memoriza los datos de entrenamiento incluyendo el ruido, perdiendo capacidad de generalización.

Subajuste

El modelo es demasiado simple para capturar los patrones importantes en los datos.

Tipos de error de un modelo

Uno de los aspectos clave en el proceso de evaluación de modelos en Machine Learning es la comprensión de los distintos tipos de errores que un modelo puede cometer. Estos errores no solo reflejan el desempeño del modelo, sino que también permiten identificar posibles problemas relacionados con el ajuste y la capacidad de generalización del modelo.

Clasificación de errores

El error de un modelo se puede clasificar principalmente en dos categorías: **Error de entrenamiento** y **Error de prueba (o generalización)**. El primero mide el desempeño del modelo sobre los datos con los que fue entrenado, mientras que el segundo evalúa su rendimiento sobre datos no vistos durante el entrenamiento, permitiendo estimar su capacidad predictiva.

Tipos generales de errores

1

Error sistemático o sesgo (Bias)

Ocurre cuando el modelo tiene suposiciones simplificadas sobre la relación entre las variables, lo que lleva a un desempeño pobre incluso en los datos de entrenamiento.

2

Error por varianza

Se genera cuando el modelo es muy sensible a pequeñas variaciones en los datos de entrenamiento, resultando en una gran fluctuación del rendimiento en diferentes conjuntos de datos.

3

Error irreducible

Proviene del ruido inherente a los datos y no puede ser eliminado por ningún modelo.

alkemy

Tabla resumen de tipos de error

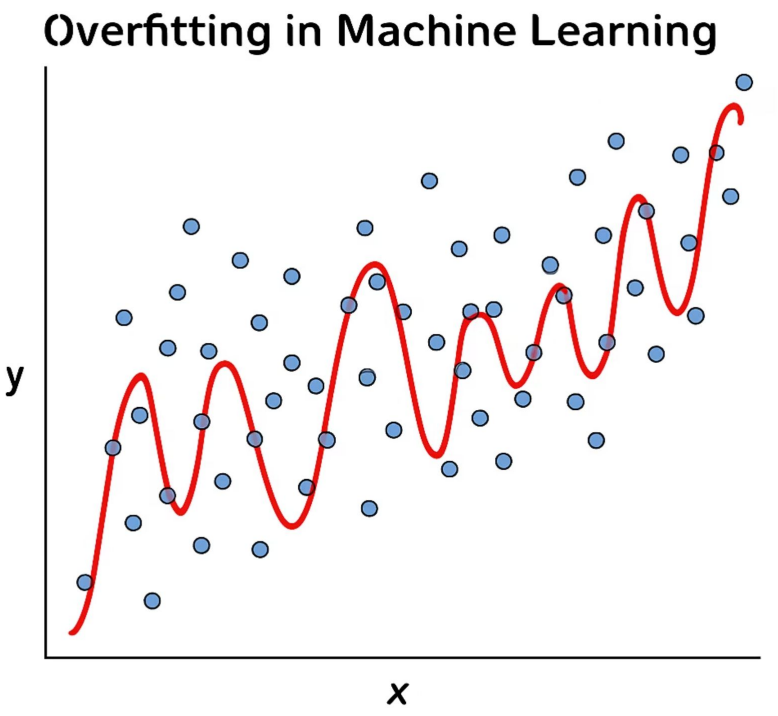
Tipo de Error	Descripción	Ejemplo Práctico
Error de entrenamiento	Diferencia entre las predicciones del modelo y los valores reales en los datos de entrenamiento.	Modelo que no logra aprender patrones básicos.
Error de prueba	Diferencia entre las predicciones del modelo y los valores reales en datos no vistos.	Modelo que funciona bien en entrenamiento, pero falla en producción.
Error de sesgo	Error debido a supuestos erróneos o simplificados.	Un modelo lineal intentando predecir datos no lineales.
Error de varianza	Error debido a la sensibilidad extrema a los datos de entrenamiento.	Árbol de decisión con demasiada profundidad.
Error irreducible	Error asociado al ruido en los datos.	Datos con mediciones erróneas o inconsistencias.

Importancia del análisis de errores

Un aspecto relevante al analizar los errores es comprender que siempre existirán, pero el objetivo es reducirlos al máximo sin comprometer la capacidad de generalización del modelo.

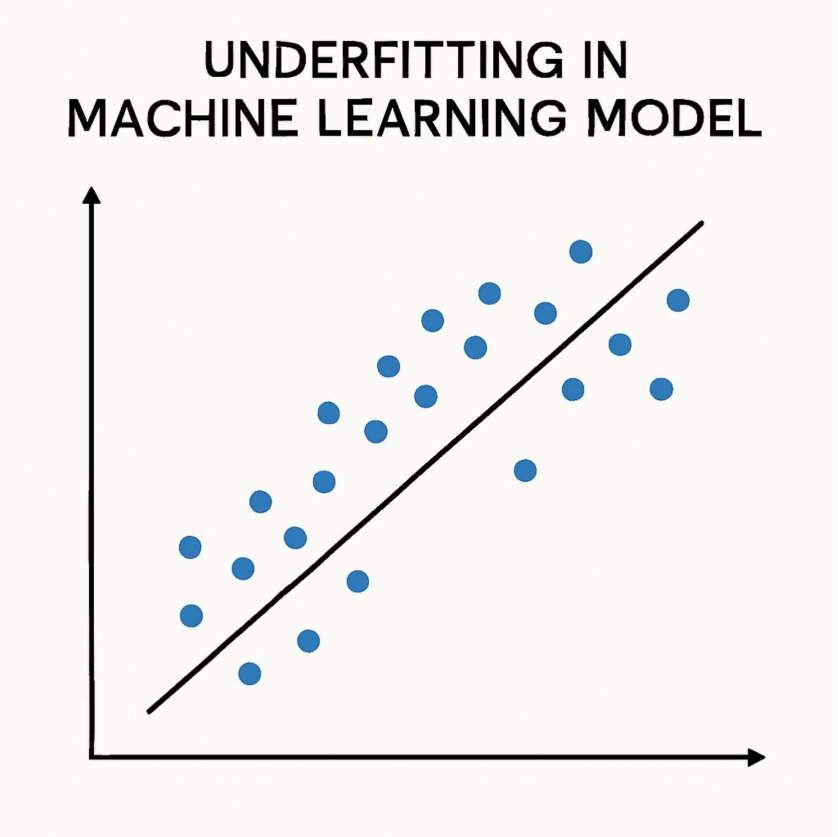
Sobreajuste, subajuste y ajuste adecuado

Cuando un modelo presenta **sobreajuste** significa que ha aprendido demasiado bien los datos de entrenamiento, incluyendo sus excepciones y ruidos. Esto se traduce en un bajo error de entrenamiento, pero un alto error en los datos de prueba. Los modelos muy complejos, como redes neuronales profundas o árboles de decisión sin poda, son susceptibles a este problema.



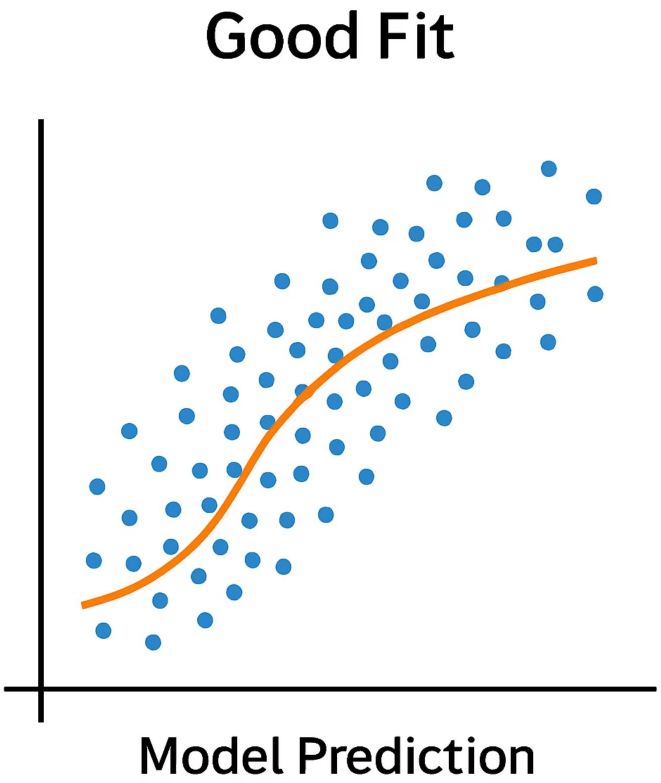
alkemy

Subajuste en modelos



Por otro lado, el **subajuste** ocurre cuando el modelo es demasiado simple para capturar los patrones subyacentes de los datos. En este caso, tanto el error de entrenamiento como el error de prueba son altos. Un ejemplo clásico sería aplicar una regresión lineal para un problema claramente no lineal.

alkemy



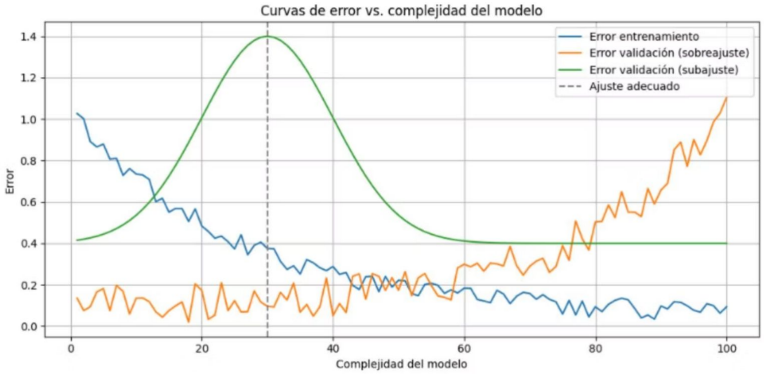
Ajuste adecuado

El objetivo siempre será lograr un **ajuste adecuado**, es decir, un modelo que no sea ni demasiado complejo ni demasiado simple. Esto permitirá un buen desempeño tanto en los datos de entrenamiento como en los datos de prueba, manteniendo un bajo error de generalización.

alkemy

Detección del nivel de ajuste

Detectar estos estados es posible mediante la observación de las curvas de error durante el entrenamiento y utilizando técnicas de validación adecuadas. El ajuste correcto se logra iterando sobre la complejidad del modelo y aplicando métodos de validación cruzada.



🧪 Ejemplo de código con validación cruzada

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.datasets import make_regression

# Generar datos simulados
X, y = make_regression(n_samples=100, n_features=1, noise=10, random_state=42)

# Modelo simple
modelo = LinearRegression()

# Validación cruzada 5-fold
scores = cross_val_score(modelo, X, y, cv=5, scoring='neg_mean_squared_error')

# Mostrar resultados
print("Errores de validación cruzada (MSE):", -scores)
print("Error promedio:", -scores.mean())
```

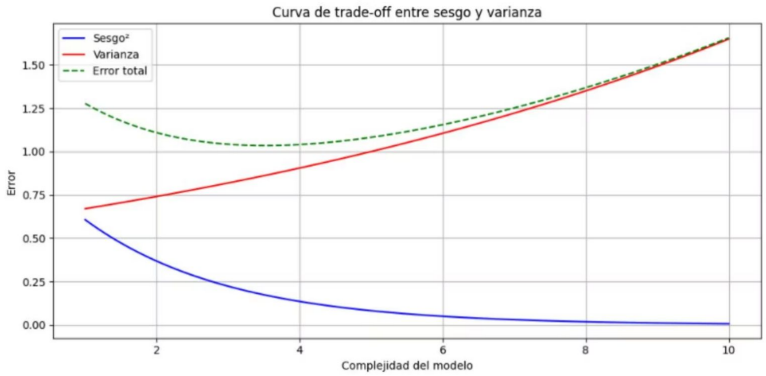
alkemy

Trade-off entre sesgo y varianza

Uno de los conceptos clave en el ajuste de modelos es el **trade-off entre sesgo y varianza**. El **sesgo** se refiere a los errores introducidos por un modelo demasiado simple, incapaz de capturar las relaciones complejas en los datos. La **varianza**, en cambio, está asociada a un modelo muy complejo que se ajusta excesivamente a los datos de entrenamiento, siendo sensible a pequeñas variaciones.

Curva de sesgo-varianza

Este equilibrio entre sesgo y varianza es representado gráficamente por la conocida **curva de sesgo-varianza**. Cuando el sesgo es alto, el modelo subestima la relación entre las variables; cuando la varianza es alta, el modelo sobreestima estas relaciones, afectando su capacidad de generalización.



alkemy

Encontrando el equilibrio

La clave está en encontrar un punto intermedio donde ambos factores estén equilibrados, minimizando el error total. Esto se logra mediante la correcta selección del modelo, la complejidad adecuada y la implementación de técnicas de regularización o validación.

Comprender este trade-off es fundamental para los científicos de datos y analistas, ya que permite tomar decisiones informadas en el proceso de ajuste y evitar problemas que podrían comprometer la utilidad del modelo.

Ejemplo de código práctico con validación cruzada

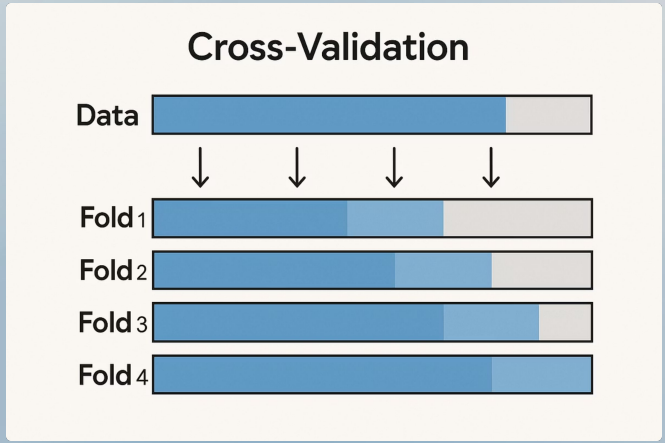
Este código muestra cómo implementar validación cruzada para evaluar el rendimiento de un modelo y evitar problemas de sobreajuste.

```python
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.model_selection import cross_val_score
import numpy as np

# Generar datos
X, y = make_regression(n_samples=100, n_features=20, noise=10, random_state=0)

# Modelo sin regularización (alta varianza posible)
modelo_simple = LinearRegression()
scores_simple = cross_val_score(modelo_simple, X, y, cv=5, scoring='neg_mean_squared_error')

# Modelo regularizado (menor varianza)
modelo_ridge = Ridge(alpha=10)
scores_ridge = cross_val_score(modelo_ridge, X, y, cv=5, scoring='neg_mean_squared_error')

print("Error promedio (LinearRegression):", -np.mean(scores_simple))
print("Error promedio (Ridge α=10):", -np.mean(scores_ridge))
```

alkemy

¿Qué es la validación cruzada?

La **validación cruzada** es una técnica estadística utilizada para evaluar el desempeño de un modelo y garantizar que los resultados obtenidos no sean producto de una partición específica de los datos. Consiste en dividir el conjunto de datos en múltiples subconjuntos, entrenando y evaluando el modelo varias veces con diferentes particiones.



alkemy

Objetivos de la validación cruzada

Estimación robusta

Obtener una estimación más robusta del error de generalización del modelo.

Detección de problemas

Detectar posibles problemas de sobreajuste o subajuste, proporcionando una visión más realista del desempeño esperado.

Comparación de modelos

Permitir la comparación objetiva entre diferentes modelos o configuraciones de hiperparámetros.

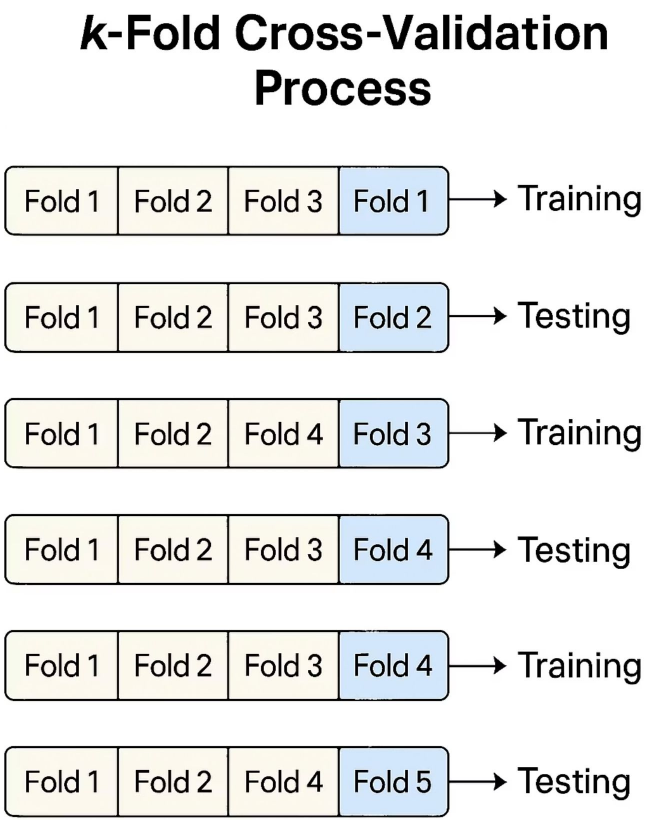
Métodos de validación cruzada

Existen diversos métodos de validación cruzada, cada uno con sus ventajas y desventajas. La elección de un método específico depende del tamaño del conjunto de datos, la complejidad del modelo y los recursos computacionales disponibles.

La implementación adecuada de la validación cruzada es esencial en cualquier proyecto de aprendizaje de máquina, ya que mejora la confiabilidad de las conclusiones y evita tomar decisiones basadas en resultados engañosos.

Introducción a la validación cruzada

La validación cruzada es una de las técnicas más utilizadas en el aprendizaje de máquina para evaluar la capacidad de generalización de un modelo. Su objetivo principal es asegurar que el modelo no se ajuste únicamente a los datos de entrenamiento, sino que también tenga un buen rendimiento con datos nuevos. A diferencia de una simple separación entrenamiento/prueba, la validación cruzada permite utilizar la totalidad de los datos para entrenar y probar el modelo de forma sistemática.



Funcionamiento de la validación cruzada

Esta técnica consiste en dividir el conjunto de datos en múltiples subconjuntos o "folds". Durante cada iteración, un fold es utilizado como conjunto de prueba y los demás como conjunto de entrenamiento. El proceso se repite tantas veces como folds existan y los resultados obtenidos se promedian para obtener una métrica más estable.

alkemy

Ventajas de la validación cruzada

Reducción de varianza

Una de las ventajas clave de la validación cruzada es que reduce la varianza de la estimación del rendimiento del modelo, ya que permite evaluar el modelo en diferentes subconjuntos de datos.

Maximización de datos

Esto es especialmente útil cuando se dispone de un conjunto de datos limitado, ya que maximiza el uso de la información disponible.

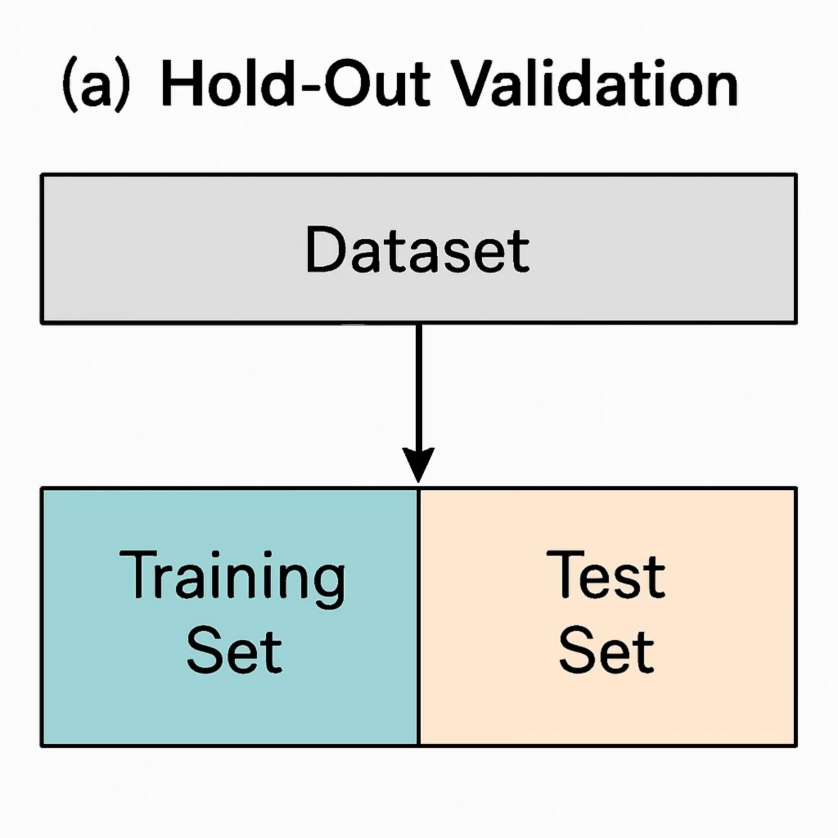
Detección de problemas

Además, la validación cruzada ayuda a detectar problemas de sobreajuste o subajuste, brindando una perspectiva más precisa sobre la capacidad de generalización del modelo y facilitando la comparación entre diferentes algoritmos o configuraciones.

alkemy

Método de Retención

El método de retención, también conocido como Hold-Out, es una de las técnicas más simples y utilizadas para validar un modelo de aprendizaje automático. Consiste en dividir el conjunto de datos en dos subconjuntos: uno para el entrenamiento del modelo y otro para la evaluación de su desempeño. Comúnmente, se utiliza un 70% u 80% de los datos para el entrenamiento y el porcentaje restante para la prueba.



alkemy

Ventajas del método de retención

Simplicidad

Este método es sencillo de implementar y rápido de ejecutar, especialmente útil cuando se cuenta con un volumen grande de datos.

Simulación de datos nuevos

La idea es que el conjunto de prueba simule datos nuevos y no vistos, permitiendo así obtener una estimación del rendimiento del modelo.

Limitaciones del método de retención

Dependencia de la partición

Una de las principales es que los resultados pueden depender excesivamente de cómo se haya realizado la partición de los datos. Si los datos no son representativos o contienen un sesgo, la evaluación será poco confiable.

Uso ineficiente de datos

Además, la partición simple no permite aprovechar al máximo los datos disponibles. Parte de la información útil para el entrenamiento se pierde al reservar un subconjunto exclusivamente para la evaluación.

Problemas con datos pequeños

Esto es especialmente problemático cuando se trabaja con conjuntos de datos pequeños.

Recomendaciones para el método de retención

Por estas razones, el método de retención se recomienda principalmente para casos exploratorios, pruebas rápidas, o cuando se dispone de un volumen masivo de datos y no se requiere una evaluación refinada.

Finalmente, para reducir la variabilidad de los resultados, es posible repetir el método varias veces utilizando diferentes particiones y promediar los resultados, aunque esta práctica ya anticipa la lógica de técnicas más avanzadas como la validación cruzada.

Live Coding

¿En qué consistirá la Demo?

Vamos a entrenar un modelo de regresión y observar su rendimiento bajo distintos métodos de validación.

1. *Cargar y explorar el dataset*
2. *Separar variables predictoras y objetivo*
3. *Entrenar un modelo con train_test_split*
4. *Evaluar métricas básicas (R^2, MAE)*
5. *Aplicar validación cruzada con cross_val_score*
6. *Visualizar la variabilidad en las métricas*
7. *Analizar cómo influye la validación cruzada en la toma de decisiones*

Tiempo: 25 Minutos

Momento:

Time-out!

⏰5 -10 min.

alkemy

Ejercicio

¿El modelo está bien ajustado?

¿El modelo está bien ajustado?

Contexto: 🙌

Vamos a evaluar el rendimiento de un modelo de regresión lineal aplicando validación cruzada, para observar cómo varía su comportamiento en distintos subconjuntos de los datos.

Consigna: ✍️

Aplique validación cruzada con KFold (k=5) sobre un modelo de regresión entrenado con el dataset Boston Housing (o el dataset que estés trabajando en clase). Luego, analizá las métricas obtenidas.

Paso a paso: ⚙️

1. Cargue el dataset (load_boston o uno propio)
2. Separe variables predictoras y target
3. Normalice los datos si es necesario
4. Cree un modelo de regresión lineal
5. Aplique validación cruzada con cross_val_score
6. Obten y graficar los scores de R^2 de cada fold
7. Calcule el promedio de R^2
8. Discute: ¿hay mucha variabilidad entre folds? ¿el modelo está bien ajustado?

Tiempo 🕛: *45 Minutos*

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ ***Comprendimos qué es el ajuste de un modelo***
- ✓ ***Identificamos tipos de errores comunes (sesgo, varianza, irreducible)***
- ✓ ***Diferenciamos sobreajuste y subajuste con ejemplos***
- ✓ ***Exploramos el trade-off entre sesgo y varianza***
- ✓ ***Implementamos validación cruzada para evaluar modelos***

¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

alkemy

< ¡Muchas gracias! >

alkemy